

206. Reverse LinkedList

Description:

- ✓ Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:

Input : 1 -> 2 -> 3 -> 4 -> 5

Output : 5 -> 4 -> 3 -> 2 -> 1

Example 2:

Input : 1 -> 2

Output : 2 -> 1

Example 3:

Input : []

Output : []

Constraints:

- a) The number of nodes in the list is the range [0, 5000].
- b) $-5000 \leq \text{Node.val} \leq 5000$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple

5

Amazon

3

Microsoft

2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg

6

Adobe

5

Oracle

4

Facebook

3

Google

3

ServiceNow

3

Uber

2

VMware

2

Nagarro

2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Nvidia 6

Yandex 6

Paypal 3

Intuit 3

eBay 3

IBM 3

Cisco 3

Yahoo 2

Samsung 2

Dell 2

PayTM 2

JPMorgan 2

HBO 2

TikTok 2

Canonical 2

Snapchat

Zenefits

Yelp

Twitter

206. Reverse LinkedList- Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode reverseList(ListNode headNode) {

        ListNode prevNode = null;
        ListNode nextNode = null;

        ListNode currNode = headNode;

        while(currNode!=null) {
            nextNode = currNode.next;
            currNode.next=prevNode;

            prevNode=currNode;
            currNode=nextNode;
        }
        return prevNode;
    }
}
```

234. Palindrome Linked List

Description:

- ✓ Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:

Input: head = [1,2,2,1]

Output: true

Example 2:

Input: head = [1,2]

Output: false

Constraints:

- a) The number of nodes in the list is in the range [1, 105].
- b) $0 \leq \text{Node.val} \leq 9$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 5

Microsoft 3

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 7

Facebook 5

Google 5

Adobe 4

Bloomberg 3

Yahoo 3

Intuit 3

Spotify 3

ServiceNow 2

Accenture 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

VMware 2

Oracle 2

Paypal 2

IXL

234. Palindrome Linked List

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {

    public ListNode middleNode(ListNode headNode) {

        ListNode slow = headNode;
        ListNode fast = headNode;

        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        return slow;
    }

    public ListNode reverseList(ListNode headNode) {

        ListNode prevNode = null;
        ListNode nextNode = null;

        ListNode currNode = headNode;

        while (currNode != null) {
            nextNode = currNode.next;
            currNode.next = prevNode;
        }
    }
}
```



```
        prevNode = currNode;
        currNode = nextNode;

    }
    return prevNode;
}

public boolean isPalindrome(ListNode headNode) {
    if (headNode == null) {
        return true;
    }

    ListNode middle = middleNode(headNode);

    ListNode afterMiddle = reverseList(middle.next);

    ListNode part1 = headNode;
    ListNode part2 = afterMiddle;
    boolean flag = true;
    while (flag && part2 != null) {
        if (part1.val != part2.val) {
            flag = false;
        }
        part1 = part1.next;
        part2 = part2.next;
    }
    return flag;
}
}
```



328. Odd Even Linked List

Description:

- ✓ Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.
- ✓ The first node is considered odd, and the second node is even, and so on.
- ✓ Note that the relative order inside both the even and odd groups should remain as it was in the input.
- ✓ You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

Example 1:

Input: head = [1,2,3,4,5] Output: [1,3,5,2,4]

Example 2:

Input: head = [2,1,3,5,6,4,7] Output: [2,3,6,7,1,5,4]

Constraints:

- a) The number of nodes in the linked list is in the range [0, 104].
- b) $-10^6 \leq \text{Node.val} \leq 10^6$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **6**

Apple **4**

Yahoo **2**

Facebook **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft **5**

Bloomberg **4**

Adobe **4**

eBay **2**

VMware **2**

Oracle **2**

TikTok **2**

328. Odd Even Linked List

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode oddEvenList(ListNode headNode) {
        if (headNode == null)
            return null;
        ListNode odd = headNode;
        ListNode even = headNode.next;
        ListNode evenHead = even;

        while (even != null && even.next != null) {
            odd.next = even.next;
            odd = odd.next;
            even.next = odd.next;
            even = even.next;
        }
        odd.next = evenHead;
        return headNode;
    }
}
```

141. Linked List Cycle

Description:

- ✓ Given head, the head of a linked list, determine if the linked list has a cycle in it.
- ✓ There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.
- ✓ Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

Input : 3 -> 2 -> 0 -> 4 -> 2 **Output :** true

Example 2:

Input : 1 -> 2 -> 1 **Output :** true

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- pos is -1 or a valid index in the linked-list.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **2**



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple **7**

Adobe **4**

Bloomberg **3**

Oracle **3**

Cisco **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft **13**

Spotify **6**

Google **5**

Visa **5**

Goldman Sachs **4**

Nvidia **4**

Facebook **3**

Yahoo **3**

Salesforce **3**

Intel **2**

Sprinklr **2**

Splunk **2**

141. Linked List Cycle- Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public boolean hasCycle(ListNode headNode) {
        if (headNode == null) {
            return false;
        }
        ListNode slow = headNode;
        ListNode fast = headNode;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                return true;
            }
        }
        return false;
    }
}
```

160. Intersection of Two Linked Lists

Description:

- ✓ Given the heads of two singly linked-lists **headA** and **headB**, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

Example 1:

Input :

headA : 4 -> 1 -> 8 -> 4 -> 5

HeadB : 5 -> 6 -> 1 -> 8 -> 4 -> 5

Output: Intersected at '8'

Input :

headA : 2 -> 6 -> 4

HeadB : 1 -> 5

Output: No intersection

Constraints:

- a) The number of nodes of listA is in the m.
- b) The number of nodes of listB is in the n.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Airbnb 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 6

Microsoft 5

Amazon 2

Bloomberg 2

Facebook 2

Oracle 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

LinkedIn 10

Adobe 6

TikTok 6

Nvidia 5

Google 2

Goldman Sachs 2

Samsung 2

Spotify 2

Nutanix 2

160. Intersection of 2 Linked Lists

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode getIntersectionNode(ListNode head1, ListNode head2) {
        ListNode currNode1 = head1;
        ListNode currNode2 = head2;
        while (currNode1 != currNode2) {
            currNode1 = (currNode1 == null) ? head2 : currNode1.next;
            currNode2 = (currNode2 == null) ? head1 : currNode2.next;
        }
        return currNode1;
    }
}
```

21. Merge Two Sorted Lists

Description:

- ✓ You are given the heads of two sorted linked lists **list1** and **list2**.
- ✓ Merge the two lists in a one sorted list. The list should be made by splicing together the nodes of the first two lists.
- ✓ Return the head of the merged linked list.

Example 1:

Input: list1 = [1,2,4], list2 = [1,3,4] **Output:** [1,1,2,3,4,4]

Example 2:

Input: list1 = [], list2 = [] **Output:** []

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in non-decreasing order.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 4 Expedia 3 Apple 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Adobe 10 Facebook 8 Google 8 Bloomberg 7 Microsoft 6
Uber 4 Yandex 3 Indeed 2 Yahoo 2 Visa 2 eBay 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

VMware 4

Oracle 4

ByteDance 4

Shopee 4

LinkedIn 2

Accenture 2

Intuit 2

Walmart Global Tech 2

GoDaddy 2

Roblox 2

Wix

21. Merge Two Sorted Lists

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {

        ListNode head = new ListNode(-1);
        ListNode tail = head;

        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                tail.next = list1;
                list1 = list1.next;
            } else {
                tail.next = list2;
                list2 = list2.next;
            }
            tail = tail.next;
        }
        tail.next = (list1 == null) ? list2 : list1;
        return head.next;
    }
}
```